

App Dev Project Report

1. Student Details

Name: << Your full name >>

Roll Number: << Roll number >>

Email: << student email >>

About Me: I am a student interested in full-stack web development and backend engineering. I enjoy creating practical systems that improve real institutional workflows through automation, data consistency, and role-based access.

2. AI/LLM Usage

AI/LLM used: GPT-5.2-Codex (OpenAI).

Extent of use: Approximately 20-30% for report drafting/formatting and documentation support. Repository understanding, architecture interpretation, and implementation details were validated manually from source files.

3. Description

This project implements a Placement Portal Application (PPA) for three roles: Admin, Company, and Student. It supports company approvals, placement drive management, student applications, status tracking, reporting, caching, and asynchronous/scheduled jobs. The design follows the mandated stack and local-run constraints provided by the institute.

AI/LLM usage percentage and extra details: About 20-30% of assistance was used for writing quality and structure in this report. The actual project analysis was completed by directly reading and summarizing repository code.

4. Technologies Used

- `Flask`
- `Flask-SQLAlchemy`
- `Flask-Migrate`
- `Flask-JWT-Extended`
- `Flask-Cors`
- `redis`
- `celery`
- `python-dotenv`
- `passlib`
- `email-validator`
- `marshmallow`
- `SQLAlchemy`

- `Vue 3 (CDN)`
- `Bootstrap`
- `Axios`
- `Jinja2 templates`

Purpose behind using these technologies:

- Flask stack provides API modularity with role-protected routes and service-layer organization.
- SQLite satisfies the mandatory local relational storage requirement.
- Redis + Celery handle cache and background workloads (daily, monthly, async export).
- Vue + Bootstrap provide responsive, role-centric user interfaces.

5. DB Schema Design

Database is created programmatically via SQLAlchemy model definitions (no manual DB creation).

Table structure, columns, and constraints

- `applications`: id (Integer; pk), student_id (Integer; not null), drive_id (Integer; not null), status (String; not null), interview_schedule (String). Constraints: unique(student_id, drive_id).
- `company\_profiles`: id (Integer; pk), user_id (Integer; not null, unique), company_name (String; not null), website (String), description (Text), approved (Boolean; not null).
- `placement\_drives`: id (Integer; pk), company_id (Integer; not null), title (String; not null), description (Text; not null), eligible_branches (String; not null), min_cgpa (Float; not null), graduation_year (Integer; not null), deadline (DateTime; not null), approved (Boolean; not null), closed (Boolean; not null).
- `student\_profiles`: id (Integer; pk), user_id (Integer; not null, unique), branch (String; not null), graduation_year (Integer; not null), cgpa (Float; not null), resume_path (String).
- `users`: id (Integer; pk), name (String; not null), email (String; not null, unique), password_hash (String; not null), role (String; not null), is_active (Boolean; not null), is_blacklisted (Boolean; not null).

Design reasons

- A unified `users` table centralizes authentication and role handling.
- Separate `student\_profiles` and `company\_profiles` normalize role-specific attributes.
- `applications` models many-to-many linkage between students and drives while also tracking lifecycle status.
- Unique and FK constraints enforce consistency and prevent invalid/duplicate operations.

6. API Design

APIs are implemented using role-based Flask Blueprints with JWT-protected access.

API modules

- `/api/auth/\*`: Authentication and account access (2 endpoints).
- `/api/admin/\*`: Admin approvals, moderation, search, analytics (9 endpoints).
- `/api/company/\*`: Company profile, drives, applicant workflow (6 endpoints).

- `/api/student/*`: Student profile, applications, exports (7 endpoints).

Endpoint list

- `GET /api/admin/companies``
- `PATCH /api/admin/companies//approval``
- `GET /api/admin/dashboard``
- `GET /api/admin/drives``
- `PATCH /api/admin/drives//approval``
- `PATCH /api/admin/drives//close``
- `GET /api/admin/reports``
- `GET /api/admin/students``
- `PATCH /api/admin/users//blacklist``
- `POST /api/auth/login``
- `POST /api/auth/register``
- `PATCH /api/company/applications/``
- `GET /api/company/drives``
- `POST /api/company/drives``
- `GET /api/company/drives//applicants``
- `PATCH /api/company/drives//close``
- `POST /api/company/profile``
- `GET /api/student/applications``
- `POST /api/student/applications/export``
- `GET /api/student/drives``
- `POST /api/student/drives//apply``
- `GET /api/student/profile``
- `POST /api/student/profile``
- `POST /api/student/resume``

YAML/API specification should be submitted separately as instructed.

7. Architecture and Features

Architecture overview

- `backend/app.py`: app factory, extension setup, blueprint registration, DB init command.
- `backend/models/`: SQLAlchemy entities for users, profiles, drives, applications.
- `backend/routes/`: controller layer grouped by role.
- `backend/services/`: business logic (approval flows, eligibility, validations).
- `backend/tasks/`: Celery tasks for reminders, monthly reports, CSV export.

- ``backend/utills/``: auth decorators, validators, cache and notification utilities.
- ``frontend/components/``: role-specific dashboards and authentication views.

Implemented features

- Admin pre-seeding and role-based authentication/login.
- Company registration approval and drive approval workflows.
- Company drive lifecycle management and application status updates.
- Student profile/resume management, eligibility filtering, apply/history flows.
- Duplicate application prevention through DB uniqueness + service checks.
- Scheduled jobs: daily reminders, monthly report generation, async CSV export.
- Redis-backed cache for selected dashboard/search/list operations.

Repository scan summary

- Total non-PDF files scanned (excluding `.git``): 49
- Backend files scanned: 31
- Frontend files scanned: 7
- Docs files scanned: 3

8. Video

Video Link (<= 3 minutes): << Add your public video URL here >>